

Scalable Backend System using Microservices Architecture (SBSMA)

Step-by-Step Execution, System Design, Database-per-Service, Dual Frontends & Verification

PLATFORM

Fashion Store & Admin

ARCHITECTURE

5 Microservices +
Gateway

TEST STATUS

30 / 30 Passed (100%)

TECH STACK

Node.js, Express, SQLite

Step 1 Part A: System Design & Microservices Architecture

We designed and implemented a fully decoupled, scalable e-commerce backend by decomposing business domains into 5 independent microservices orchestrated via an API Gateway:

Service Name	Port	Database	Primary Responsibilities
API Gateway	:3000	None (Stateless)	Reverse proxy routing, Unified Swagger UI (/api-docs), health aggregation, serves Fashion customer UI and Admin portal.
User Service	:3001	users.db	User registration, profiles, password hashing, role management, address book CRUD.
Product Service	:3002	products.db	Product catalog, category hierarchy, SKU validation, atomic stock reservation & rollback.
Order Service	:3003	orders.db	Order placement, order items lifecycle, inter-service inventory reservation with Product Service.
Payment Service	:3004	payments.db	Simulated multi-channel checkout (UPI, Card, COD), transaction tracking, order status confirmation.
Notification Service	:3005	notifications.db	Asynchronous event alerts (Email, SMS, in-app notifications) for orders and billing.

[Client / Browser / Postman / Swagger]

▼ HTTP Port 3000 (API Gateway)

```
/api/users      → Proxy to User Service (:3001) |
/api/products  → Proxy to Product Service (:3002) |
/api/orders    → Proxy to Order Service (:3003) |
/api/payments  → Proxy to Payment Service (:3004) |
/api/notifications→ Proxy to Notification Serv (:3005) |
```

Step 2 Part B: Database-per-Service Design

Following strict microservices best practices, no microservice shares a database. Each service owns its private SQLite database file:

Database File	Tables	Primary Key (PK) & Foreign Keys (FK)
users.db	users, addresses	users.id (PK, Auto-inc), addresses.id (PK), addresses.user_id (FK -> users.id ON DELETE CASCADE)
products.db	categories, products	categories.id (PK), products.id (PK), products.category_id (FK -> categories.id), products.sku (UNIQUE)
orders.db	orders, order_items	orders.id (PK), orders.order_number (UNIQUE), order_items.id (PK), order_items.order_id (FK -> orders.id)
payments.db	payments	payments.id (PK), payments.transaction_id (UNIQUE), payments.order_id, payments.user_id
notifications.db	notifications	notifications.id (PK), notifications.user_id, notifications.channel, notifications.status

Step 3 How Microservices Communicate with Each Other (Inter-Service Flow)

Since services are decoupled, they communicate over the local network using **RESTful HTTP calls with JSON payloads** using the built-in `fetch()` API.

Primary Example: Order Service (:3003) → Product Service (:3002)

When a customer orders an item, Order Service does NOT touch `products.db`. Instead:

1. **Verify Item & Price:** GET `http://localhost:3002/api/products/:id`
2. **Atomic Stock Reservation:** POST `http://localhost:3002/api/products/:id/reserve-stock` with `{ quantity }`
3. **Compensating Rollback (Saga Pattern):** If any item has insufficient stock, Order Service calls POST `http://localhost:3002/api/products/:id/restore-stock` to return previously reserved inventory.

```
// Code from services/order-service/server.js
const reserveRes = await fetch(`${PRODUCT_SERVICE_URL}/api/products/${item.product_id}/reserve-stock`,
{
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ quantity: item.quantity })
});

if (!reserveRes.ok) {
  // Compensating transaction (Rollback)
  for (const resItem of reservedItems) {
    await fetch(`${PRODUCT_SERVICE_URL}/api/products/${resItem.product_id}/restore-stock`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ quantity: resItem.quantity })
    });
  }
  return sendError(res, 'Insufficient stock', 400);
}
```

Secondary Example: Payment Service (:3004) → Order Service (:3003)

Once payment is confirmed in `payments.db`, Payment Service updates Order Service via: PUT `http://localhost:3003/api/orders/:id/status` with payload `{ status: "CONFIRMED" }`.

Frontend Architecture: Customer Storefront vs. Admin Portal

To provide a realistic e-commerce experience, we separated the customer shopping interface and the merchant administration portal:

Aspect	Customer Storefront (/)	Admin Management Portal (/admin)
URL Path	http://localhost:3000/	http://localhost:3000/admin
Branding Name	fashion (TRENDS)	Fashion Admin Portal
Admin Link Visibility	COMPLETELY HIDDEN (Zero admin buttons, links, or logins)	Dedicated URL access for site managers
Customer Features	Browse categories, live search, product cards (discount badge, free delivery, rating), slide-out shopping cart drawer, multi-step checkout.	Catalog CRUD (Add product, live price edit, +50 quick restock, delete), order status fulfillment, users view, transaction logs.
Real-Time Sync	Adding or modifying any product in the Admin Portal immediately reflects on the Fashion storefront via Product Service (:3002).	

Step 5**Part D: Standardized APIs & Swagger Documentation**

Every API in the system complies with standard JSON response envelopes and standard HTTP status codes (200, 201, 400, 404, 409, 500):

```
// Unified JSON Response Format
{
  "success": true,
  "statusCode": 200,
  "message": "Product retrieved successfully",
  "data": {
    "id": 2,
    "name": "Kanjeevaram Jacquard Woven Soft Silk Saree",
    "price": 799,
    "stock_quantity": 80,
    "sku": "SKU-SAREE-02"
  },
  "timestamp": "2026-09-20T11:40:06.000Z"
}
```

A unified **OpenAPI / Swagger UI** is hosted live at <http://localhost:3000/api-docs>, enabling interactive testing of all endpoints across all 5 microservices.

Step 6**Part E: Automated Integration Testing (30/30 Tests Passed)**

We created an automated end-to-end integration test runner in `tests/run-all-tests.js` that exercises every service, HTTP method, and inter-service transaction:

Test Suite	Port Tested	Operations Verified	Result
Gateway Health	:3000	GET /health Aggregates all 5 services status	✓ PASS (1/1)
User Service	:3001	CRUD Users, Add Address, Delete User, 404 validation	✓ PASS (7/7)
Product Service	:3002	List catalog, categories, create product, price update, delete	✓ PASS (7/7)
Order Service	:3003	Create order, inter-service stock check, status update to SHIPPED	✓ PASS (4/4)
Payment Service	:3004	Process transaction, order confirmation trigger, refund payment	✓ PASS (5/5)
Notification Service	:3005	Dispatch alert, fetch alert, mark read, delete alert	✓ PASS (5/5)
Total Suite	:3000 - :3005	Full CRUD & Inter-Service Workflows	30 / 30 PASSED (100%)

Step 7 Execution Guide: Commands to Run in CMD

To run the entire system on Windows Command Prompt (cmd):

```
:: Step 1: Navigate to the project directory
cd /d "c:\microservic project"

:: Step 2: Start all 5 microservices & API Gateway simultaneously
npm start
:: (or: node start-all.js)

:: Step 3: Run all 30 automated integration tests
npm test
:: (or: node tests/run-all-tests.js)
```

Fashion Store: <http://localhost:3000>

Admin Portal: <http://localhost:3000/admin>

Swagger UI: <http://localhost:3000/api-docs>